```
2 Pregunta 1 — Convolución a mano (3×3)
═══════════════════════════════════════════════

  Tienes este trozo de imagen en escala de grises (un canal) y este
  kernel:

   ┌─────
   |   Imagen 5×5                Kernel 3×3
   |   0  0  1  0  0             1  0 -1
   |   0  1  2  1  0             1  0 -1
   |   1  2  3  2  1             1  0 -1
   |   0  1  2  1  0
   |   0  0  1  0  0
   └─────


2.0.1 Tareas
------------

  1. Con *stride = 1* y *padding valid* (sin relleno), ¿cuál es el
     tamaño del mapa de salida?
  2. Calcula *solo* el valor de la esquina superior izquierda del mapa
     (la primera posición del filtro). Muestra la suma de productos.
  3. ¿Este kernel parece detectar bordes *verticales* u *horizontales*?
     Justifica mirando los signos.
```

1.-Formula para Sacar el Mapa de Salida [(N-K)/S]+1
N=5
S=1 (Stride)
K=3
Respuesta = [(5-3)/1]+1 = 3 Entonces mapa de salida 3 x 3

2.- Multiplicación

- (0 x 1) + (0 x 0) + (1 x -1) = -1
- (0 x 1) + (1 x 0) + (2 x -1) = -2
- (1 x 1) + (2 x 0) + (3 x -1) = -2
- Total = -1 -2 -2 = -5

3.- El Kernel detecta bordes verticales ya que al tener valores positivos en la primera columna, ceros en la central y negativos en la derecha, calcula la diferencia de intensidad entre los píxeles de la izquierda y los de la derecha.

```
3 Pregunta 2 — Contar pesos
═══════════════════════════

  Una imagen RGB de *28×28×3*. Compara dos modelos:

  • *Modelo A (MLP ingenuo):* aplastas la imagen a un vector y conectas
    *todas* las entradas a una capa densa de *64 neuronas* (sin bias por
    ahora).
  • *Modelo B (CNN):* una `Conv2D' con *16 filtros* de *3×3*,
    `padding''same'=, sobre la misma imagen (con bias: un bias por
    filtro).


3.0.1 Tareas
────────────

  1. ¿Cuántos pesos tiene la capa densa del Modelo A? (fórmula y número)
  2. ¿Cuántos parámetros tiene la Conv2D del Modelo B? (pesos de kernels
     + biases)
  3. En *una frase*: ¿por qué la CNN suele generalizar mejor con el
     mismo número de imágenes de entrenamiento?
```

1.- La imagen aplastada tiene 28 x 28 x 3 = 2352 valores eso quiere decir que al conectarlo con las 64 neuronas bias el total de pesos es de 2352 x 64 = 150 528 .

2.-Cada uno de los 16 filtros tiene 3 x 3 x 3 canales de profundidad osea 27 sumando el bias por cada filtro nos daría (27 x 16) + 16 = 448 parmetros

3.-La CNN generaliza mejor porque utiliza pesos compartidos e interacciones locales, reduciendo drásticamente la cantidad de parámetros para aprender y evitando el sobreajuste que sufre el MLP al conectar cada píxel individualmente.

```
4 Pregunta 3 — ¿Qué cambia si meto más capas?
═════════════════════════════════════════════

  Parte del notebook `CNNriesgo' (simplificado):

  ┌──────
  │   Entrada (21×28×3)
  │     → Conv2D(32, 3×3, same) + LeakyReLU
  │     → MaxPool(2×2)
  │     → Flatten → Dense(32) → Dense(nClasses, softmax)
  └──────

  Quieres *apilar un segundo bloque* convolucional *antes* del Flatten:

  ┌──────
  │   … → Conv2D(32) → Pool → Conv2D(64, 3×3, same) → Pool → Flatten → …
  └──────


4.0.1 Tareas
────────────

  1. Tras el *primer* MaxPool(2×2) con padding same sobre 21×28, ¿qué
     tamaño aproximado (alto×ancho) esperas? (puedes redondear / usar la
     regla “casi la mitad”)
  2. ¿Qué *significan* ahora los 64 filtros de la segunda Conv2D:
     detectan bordes “crudos” o patrones *más compuestos*? Explica con
     la analogía de la lupa / capas abstractas.
  3. Riesgo: si añades *muchas* capas Conv+Pool con pocas imágenes, ¿qué
     falla típico verías en las curvas train vs validation? ¿Qué harías
     primero para mitigarlo (Dropout, más datos, menos filtros…)? Elige
     *una* medida y justifica.
```

1.- Dividiendo las dimensiones a la mitad daria un tamaño aproximado de 10 x 28

2.-En capas más profundas, los filtros detectan patrones más compuestos y abstractos (como texturas complejas, esquinas o formas como ojos u orejas). Al igual que usar una lupa para alejarse, combinan las líneas y bordes extraídos por la primera capa.

3.-El principal fallo típico en las curvas sería sobreajuste : alta precisión en train y baja en validation. Una medida inmediata es aplicar Data Augmentation (rotaciones, zoom, recortes) para forzar al modelo a aprender características invariantes multiplicando artificialmente la diversidad de las pocas imágenes originales.

```
5 Pregunta 4 — Formas: de volumen a vector
═══════════════════════════════════════════

  Supón que, justo antes del Flatten, el tensor tiene forma `(7, 7, 64)'
  (lote omitido: una sola imagen).


5.0.1 Tareas
------------

  1. ¿Cuántos números salen del Flatten? (un solo entero)
  2. Si la siguiente Dense tiene *32 neuronas* (con bias), ¿cuántos
     parámetros tiene *esa* Dense?
  3. Si en vez de Flatten usas *Global Average Pooling* (un promedio por
     canal), ¿qué forma tendrías a la salida y cuántos valores serían?
     ¿Ventaja frente a Flatten cuando la imagen de entrada es más
     grande?
```

1.- Numero que salen de Flatten es de 7 x 7 x 64 = 3 136 números que salen

2.-Cada una de las 32 neuronas se conecta a las 3,136 entradas, más un bias por neurona: eso quiere decir que (3 136 x 32) + 32 = 100 384 parámetros

3.-La forma de salida sería un vector de 64 valores (un único promedio por cada mapa de características). La ventaja seria es que elimina por completo el riesgo de explosión de parámetros del Flatten, permitiendo además que el modelo reciba imágenes de cualquier resolución sin que la capa Dense colapse por incompatibilidad de tamaño.

```
6 Pregunta 5 — Softmax y el error “casi seguro”
═══════════════════════════════════════════════

  Una CNN de 3 clases (inundación, incendio, tornado) saca logits que,
  tras softmax, dan:

   ┌──────
   │   P = [0.10, 0.05, 0.85]
   └──────

  La etiqueta verdadera (one-hot) es `[0, 1, 0]' (incendio).


6.0.1 Tareas
------------

  1. ¿Qué clase predice el modelo (`argmax')?
  2. ¿Está “seguro” o “confundido”? Usa los números, no adjetivos
     vacíos.
  3. Sin calcular la fórmula exacta: argumenta si la *categorical
     crossentropy* de este ejemplo será *alta* o *baja*, y por qué
     (pista: castiga cuando la probabilidad de la clase correcta es
     pequeña).
```

1.- El modelo predice la tercera clase (índice 2), que corresponde a tornado.

2.-El modelo esta muy seguro , pero creo que esta algo equivocado por que deja 85% a la probabilidad de clase erronea y solo un 5% a la clase real.

3.- Será muy alta. Esta función de pérdida penaliza fuertemente cuando el modelo asigna una probabilidad muy baja a la etiqueta correcta. Como la probabilidad dada al incendio fue apenas de 0.05, el castigo numérico se dispara.

7 Pregunta 6 — Padding, stride y “se me achicó la imagen”

Entrada *32×32*, kernel *5×5*, *un* filtro, un canal.

7.0.1 Tareas

Completa la tabla (escribe el alto×ancho de salida):

padding	stride	salida (alto×ancho)
valid	1	
valid	2	
same	1	

Luego: si encadenas *tres* MaxPool(2×2) seguidos sobre una imagen 32×32 (sin cuidar bordes), ¿qué problema práctico aparece para clasificar objetos *pequeños* o detalles finos?

S = 2 - [(32 - 5)/2] + 1 = [13.5] + 1 = 14 Eso quiere decir que se mantiene la dimension original

Si encadeno seria 32 x 32 a 16 x 16 a 8 x 8 a 4 x 4 entonces el problema es que los objetos pequeños y detalles finos desaparecen o colapsan visualmente al perder tanta resolución, volviendo imposible su clasificación.

8 Pregunta 7 — Dropout: ¿dónde y para qué?

En `CNNriesgo' hay `Dropout(0.5)' después del pooling y otra vez antes de la salida.

8.0.1 Tareas

1. Durante *entrenamiento*, ¿qué hace Dropout(0.5) en una capa densa de 32 neuronas (en promedio cuántas “sobreviven” por paso)?
2. Durante *inferencia* (`predict'), ¿se apagan neuronas al azar? ¿Por qué sí o no?
3. Un compañero pone Dropout *solo* en la última época “para que no se sobreajuste al final”. ¿Tiene sentido? Critica la idea en 3–5 líneas.

1.- En promedio, 16 neuronas permanecen activas o sobreviven por cada paso en la capa densa de 32 neuronas.

2.- No se apagan neuronas al azar. Todas actúan simultáneamente, pero sus valores de salida se escalan (multiplicándose por el factor de retención) para compensar el exceso de señal respecto al entrenamiento.

3.- La idea no tiene sentido. El sobreajuste es producto de la "coadaptación" acumulativa de los pesos durante todo el proceso de optimización. Poner Dropout al final no puede "deshacer" una dependencia en los pesos que ya se consolidó durante docenas de épocas previas.

```
9 Pregunta 8 — Diseño: 5 clases de animales
═══════════════════════════════════════════════════

  Debes adaptar `CNNriesgo' a: ranas, arañas, monos, ballenas, pájaros.


9.0.1 Tareas
────────────

  1. ¿Cuántas neuronas debe tener la *última* Dense y qué activación?
     ¿Por qué?
  2. Si todas las fotos entran a *224×224×3* pero el modelo tiene
     `input_shape=(21,28,3)', ¿qué *debe* pasar antes del `fit'? (una
     frase operativa)
  3. Propón *una* mejora de arquitectura (segundo Conv, BatchNorm, data
     augmentation…)  y di *qué síntoma* en train/val te haría pensar que
     esa mejora era necesaria.
```

1.- En la ultima dense debe tener 5 neuronas con activación softmax. Esto genera 5 salidas mutuamente excluyentes que representan la distribución de probabilidad (sumando 1) para decidir entre las 5 especies.

2.- Se debes redimensionar todas las imágenes desde 224×224 al formato $(21, 28, 3)$ que exige el input_shape del modelo, o reconstruir el modelo cambiando la forma de entrada.

3.-Agregar Batch Normalization después de las capas Conv2D. Esta mejora sería necesaria si en train notas que la pérdida fluctúa salvajemente de época a época o el modelo se estanca (vanishing/exploding gradients) negándose a aprender desde las primeras iteraciones.

```
10 Pregunta 9 — CNN vs MLP vs RNN (mismo problema, mala herramienta)
═══════════════════════════════════════════════════════════════════════

  Tres escenarios. Elige *CNN*, *MLP* o *RNN/Transformer* y justifica en
  *una* frase cada uno:

  1. Predecir si un alumno aprueba con promedio, faltas y horas de
     estudio (tabla).
  2. Clasificar fotos de semáforo (rojo / amarillo / verde).
  3. Autocompletar la siguiente línea de código dado el prefijo escrito.

  Si aplastas la foto del semáforo y usas un MLP "grande", ¿puede
  *aprender*? ¿Qué sacrificas respecto a una CNN? (parámetros, datos,
  invariancia…)
```

1.- MLP, porque los datos tabulares no tienen dependencias espaciales ni temporales que se deban extraer.

2.- CNN, porque detecta jerarquías espaciales locales (formas redondas, distribución de color) sin importar su posición exacta en la foto.

3.- RNN/Transformer, porque el código es una secuencia donde el contexto temporal o previo determina exactamente qué sigue.

Sí puede aprender a base de memorizar píxeles brutos, pero sacrificas la invariancia espacial: si un semáforo aparece un poco más a la izquierda, el modelo se romperá, además de requerir cantidades insostenibles de memoria y parámetros.

```
11 Pregunta 10 — Experimento mental: mover un gato
═══════════════════════════════════════════════════════════════

  Entrenaste una CNN que detecta "hay un gato" en fotos 64×64. En el
  test, el mismo gato aparece *corrido 10 píxeles a la derecha*, mismo
  fondo, misma escala.


11.0.1 Tareas
─────────────

  1. ¿Por qué una CNN tiene *más chance* de seguir acertando que un MLP
     que ve el vector aplastado? (pesos compartidos / detección local)
  2. Si en vez de correrlo lo *giraras 90°* y nunca viste gatos de lado
     en el train, ¿la CNN "mágica" garantiza acierto? ¿Qué haría falta
     (augmentation, más datos…)?
  3. Escribe *una* pregunta que tú le harías a un compañero para
     comprobar si entendió qué son los *pesos del kernel* (no copies las
     de arriba).
```

1.- Porque aplica pesos compartidos (filtros) que barren toda la imagen. Si el patrón del gato se desplaza a la derecha, el mismo filtro lo detectará al pasar por esa nueva región, logrando invariancia a la traslación.

2.- La CNN no garantiza acierto porque las operaciones de convolución estándar no son rotacionalmente invariantes. Para que detecte el gato de lado, hace falta aplicar Data Augmentation con rotaciones aleatorias durante el entrenamiento para que aprenda el aspecto de esas características giradas.

3.- Si duplicamos el tamaño de alto y ancho en todas las imágenes de entrada de nuestra base de datos, ¿qué debes cambiar en la declaración (parámetros) de una capa Conv2D para que el modelo siga compilando y entrenando?" Respuesta Nada, los pesos del kernel no dependen del tamaño de la imagen, operan de forma local.